

AEM Code Sync application

In the EDS world there are three main pieces:

- **AEM (Document Authoring / Sites)** – your authors edit pages.
- **GitHub repo** – your blocks/, styles/, scripts/, etc., plus content files (plain HTML/Markdown).
- **Edge Delivery runtime** – Fastly + EDS runtime that serves the site.

The **AEM Code Sync app** (or “AEMCoder / Code Sync”) is the *bridge*:

- **Direction 1 – AEM → Git:** When authors edit pages in AEM (or DA.live), the app can **export/sync** that content into the repo as plain html or structured files.
- **Direction 2 – Git → AEM / local dev:** For local development, your **local CLI / dev tool** pulls content from AEM, and the browser loads **JS/CSS directly from your repo** (or from the local dev server), so you can see code changes instantly.

From a developer perspective: you code in VS Code, push to Git, and the sync app / tooling ensures that **content and code stay aligned** between AEM and the repo.

How it reads and syncs code & content

Content side: AEM ↔ Repo

When you “sync content” in EDS (via AEM Sync Code app)

- **Read phase (AEM → tool)**
 - The tool calls special **plain.html** endpoints, e.g.:
https://main--<repo>--<org>.aem.page/<path>/home.plain.html
 - That gives a **clean, block-annotated HTML representation** of the AEM page.
- **Write phase (tool → Git)**
 - The tool writes that HTML into your repo at the appropriate path (/docs/... or /pages/... depending on your project structure).
 - Commits + pushes it to Git (either automatically or when you confirm).

Code side: Repo-first

For code (blocks, styles, scripts):

- The EDS model is **code-first from the repo**:
 - You edit blocks/hero/hero.js, styles.css, etc.
 - Edge Delivery (and your local dev proxy) always read these files straight from the repo (or from a local dev server mirroring the repo contents).
- The AEM Code Sync app does **not** “execute code inside AEM”; it:
 - Uses AEM content APIs (.html/.json/.plain.html) to fetch content.
 - Keeps the repo’s content files in sync with that content.
 - Leaves execution of code to **Edge Delivery runtime** and the **browser**.

So, the app’s job is **IO and mapping**, not interpretation or execution.

“How it checks and pushes to the edge pipeline”

Complete chain from Code Sync to edge, the internal sequence looks like this:

Step 1 – App detects or receives changes

- You modify content in AEM or trigger a migration step.
- Code Sync:
 - Fetches .plain.html from AEM,
 - Updates the corresponding file(s) in your **local repo clone**.

Step 2 – App validates + prepares repo

Inside the app process:

- Runs **structure and linkage checks** on the *local* files it just wrote.
- Validates:
 - Paths,
 - Block markers,
 - Optionally lightweight HTML rules specific to your project.

If something’s wrong, it surfaces errors in the UI rather than pushing.

Step 3 – App calls Git locally

When you accept/apply in the Code Sync UI:

- App runs (internally) operations equivalent to:
- `git add <changed files>`
- `git commit -m "Sync AEM page X" # optional`
- `git push origin <branch>`
- Or skips commit and only adds/updates files if you prefer manual git.

Key point: these are **local Git operations**; **no EDS APIs are being called yet.**

Step 4 – Git **push** triggers EDS pipeline (**external to the app**)

Once the git push is accepted:

- **GitHub** (or Git provider) notifies whatever is configured:
 - E.g. EDS's managed hooks, your GitHub Actions, etc.
- They:
 - Pull the latest commit(s),
 - Run any configured validations (lint, tests),
 - Mark the build as successful/failed.
- On success, the **Edge Delivery runtime** refreshes its view of the repo and cache.

The app has **no special back door** into EDS; it relies entirely on “**push to branch → pipeline automatically reacts**”, as is standard for Edge Delivery projects.

Additionally

How the code gets executed in Edge Delivery

Once your repo is in the right shape (blocks, styles, scripts, content), this is what happens at runtime:

1. **User hits a URL** like
`https://main--<repo>--<org>.aem.page/path/page`
2. The request goes through **Fastly / EDS runtime**, which:
 - Fetches the **HTML/Markdown content file** (synced from AEM).
 - Loads the **project JS/CSS** from the repo (or the deployed artifact).
3. In the browser:
 - The generic EDS loader JS scans the DOM for **blocks** (e.g. `<div class="block hero">`).
 - For each block, it loads the corresponding JS module, e.g.:
`blocks/hero/hero.js`
 - Calls that module's default function with the block DOM node, so your code can enhance it.

The **AEM Code Sync app** is out of the picture at this point; its work happened earlier (syncing content and ensuring files are stored in Git with the right paths).

What you *won't* see the app doing internally

To be explicit, the AEM Code Sync **app does not**:

- Call Cloud Manager APIs to run an AEM pipeline.
- Call some secret “deploy edge now” HTTP endpoint with your code bundle.
- Install or activate code inside an AEM JVM.

It:

- Talks to AEM/DA for **content**,
- Talks to **Git** for repo operations,
- Relies on **EDS's Git-based integration** to push everything to the edge.

Important

The **AEM Code Sync app for Edge Delivery Services** is primarily a **content ↔ Git synchronization tool**, not a general CI/CD orchestrator. When you commit code to Git:

- The **Code Sync app does not “push code into the edge pipeline”** in the same sense Cloud Manager does for AEM Sites.
- Instead, once your repo is in a valid state, **Edge Delivery’s own CI/edge pipeline** (GitHub hooks, Fastly/EDS infra) takes over and builds/serves the site.
- **Code Sync’s role is to update the currently checked-out branch’s files and help you push.**
- **GitHub + EDS project configuration** decide what happens when that branch receives a push – and this is where feature/* vs main matters.